

書架建構工具：使用 Gemini 為 Gemini 應用程式建構 Java Cloud Function

來源：<https://codelabs.developers.google.com/bookshelf-duetai-bigquery?hl=zh-tw>

步驟數：12

我們會在 Cloud 函式中，使用 Vertex AI 生成式 AI (Gemini) 建立書籍推薦和摘要應用程式，做為 BigQuery 的遠端函式。

目錄

1. 簡介
2. 使用 Gemini 在 Java Cloud Function 上以無伺服器方式建構生成式 AI 應用程式
3. 需求條件
4. 啟用 Gemini for Google Cloud 和必要的 API
5. 使用 Gemini 準備 Cloud Functions 範本
6. 新增依附元件
7. 修改函式進入點和類別名稱
8. 建立呼叫 Gemini Pro 的方法
9. 部署 Cloud 函式
10. 呼叫已部署的 Cloud 函式
11. 清除所用資源
12. 恭喜

1. 簡介

你喜歡深入瞭解書籍，但選擇太多讓你不知從何下手嗎？想像一下，如果有一款 AI 輔助應用程式，不僅能推薦最適合你的讀物，還能根據你選擇的類型提供簡要摘要，讓你一窺書中精髓。在本程式碼研究室中，我將逐步說明如何使用 BigQuery 和 Cloud Functions (由 Gemini 提供技術支援) 建構這類應用程式。

專案總覽

我們的用途圍繞以下 4 個主要元件：

- 書籍資料庫：網際網路封存書籍的龐大 BigQuery 公開資料集，將做為我們完整的書籍目錄。
- AI 摘要引擎：Google Cloud Functions 會搭載 Gemini Pro 語言模型，根據使用者要求生成深入的摘要。
- BigQuery 整合：BigQuery 中的遠端函式，可呼叫我們的 Cloud Function，依需求提供書籍摘要和主題。
- 使用者介面：託管在 Cloud Run 的網頁應用程式，可供使用者查看結果。

我們會將實作項目分成 3 個程式碼研究室：

程式碼研究室 1：使用 Gemini 建構 Gemini 應用程式的 Java Cloud Function。

程式碼實驗室 2：使用 Gemini 搭配 BigQuery，建構僅限 SQL 的生成式 AI 應用程式。

程式碼實驗室 3：使用 Gemini 建立可與 BigQuery 互動的 Java Spring Boot 網頁應用程式。

步驟 2 · 約 0 分鐘

2. 使用 Gemini 在 Java Cloud Function 上以無伺服器方式建構生成式 AI 應用程式

建構項目

您將建立

- Java Cloud Functions 應用程式會實作 Gemini 1.0 Pro，以 JSON 陣列的形式接收特定提示做為輸入內容，並傳回回應 (標示為「replies」的 JSON 值)。
 - 您將在 Gemini 的協助下完成建構及部署步驟
-

步驟 3 · 約 0 分鐘

3. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
- 已啟用計費功能的 Google Cloud 專案

事前須知如下：

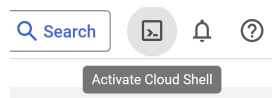
建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟用 Cloud Shell

1. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，已預先載入 **bq**：

在 Cloud 控制台，按一下右上角的「啟用 Cloud Shell」：



2. 連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

3. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案

```
gcloud config list project
```

4. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 4 · 約 0 分鐘

4. 啟用 Gemini for Google Cloud 和必要的 API

啟用 Gemini

1. 前往 [Marketplace](#) 中的 [Gemini for Google Cloud](#) 啟用 API。您也可以使用下列指令：

```
gcloud services enable cloudaicompanion.googleapis.com --project PROJECT_ID
```

2. 前往 Gemini [頁面](#)，然後按一下「開始對話」。

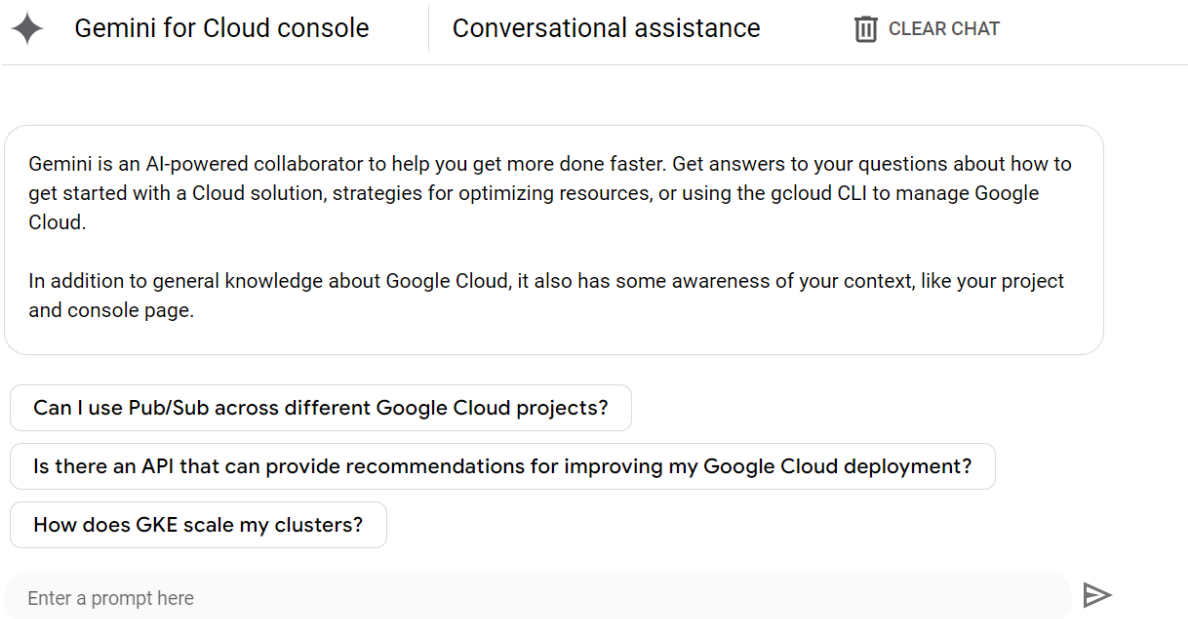
重要事項：請按照這個[程式碼實驗室](#)的步驟 1 和 2 操作，分別開始使用 Gemini 和在 [Cloud Shell IDE](#) 中啟用 Gemini。

啟用其他必要的 API

我們該怎麼做？我們來問問 Gemini 吧！但請先注意以下事項：

LLM 屬於非確定性模型，因此，您嘗試這些提示時，收到的回覆可能與我螢幕截圖中的回覆不同。

在 Google Cloud 控制台的搜尋列旁，按一下右上角的「開啟 Gemini」圖示，前往 Gemini 對話控制台。



在「在這裡輸入提示」部分輸入以下問題：

```
How do I enable the cloud functions api using a gcloud command?
```

您應該會得到類似以下的回覆：

```
gcloud services enable cloudfunctions.googleapis.com
```

複製該指令 (您可以使用指令程式碼片段頂端的複製圖示)，並在 [Cloud Shell 終端機](#) 中執行，即可啟用 Cloud Functions。對 Cloud Run 執行相同操作，因為我們需要這兩項服務，才能建構及部署 Cloud Functions：

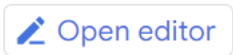
```
gcloud services enable \  
  cloudfunctions.googleapis.com \  
  aiplatform.googleapis.com \  
  run.googleapis.com \  
  cloudbuild.googleapis.com
```

步驟 5 · 約 0 分鐘

5. 使用 Gemini 準備 Cloud Functions 範本

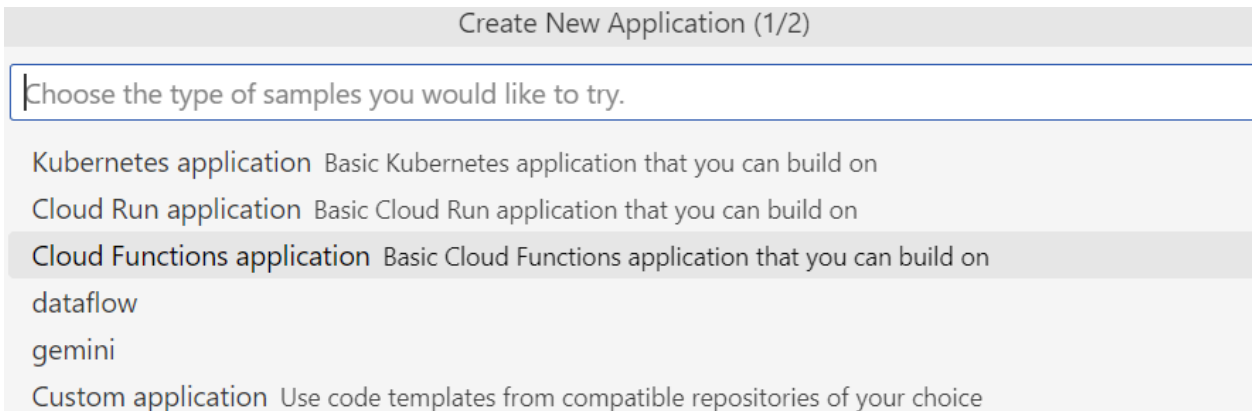
此時，我假設您已在 Cloud Shell IDE 中啟用 Gemini。

點按 Cloud Shell 終端機右上角的「開啟編輯器」圖示，開啟 Cloud Shell 編輯器 (我通常偏好在不同的分頁中並行開啟終端機和編輯器，這樣就能在一個分頁中編寫程式碼，在另一個分頁中建構程式碼)。

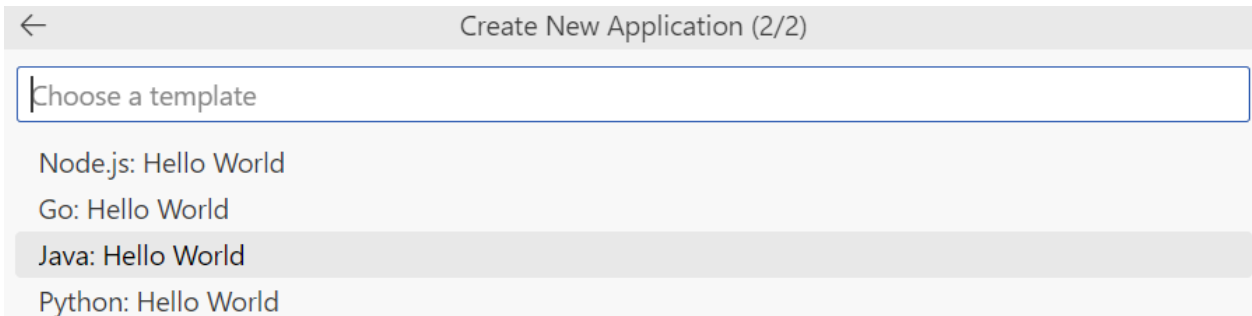


開啟編輯器後，請確認編輯器控制台右下角的 Gemini 標誌已啟用 (而非取消)。此外，請確認左下角的 Google Cloud 專案指向您目前要使用的有效專案。如果這些服務處於非啟用狀態，請點選服務、授權，然後選取要指向的 Google Cloud 專案，並啟用服務。

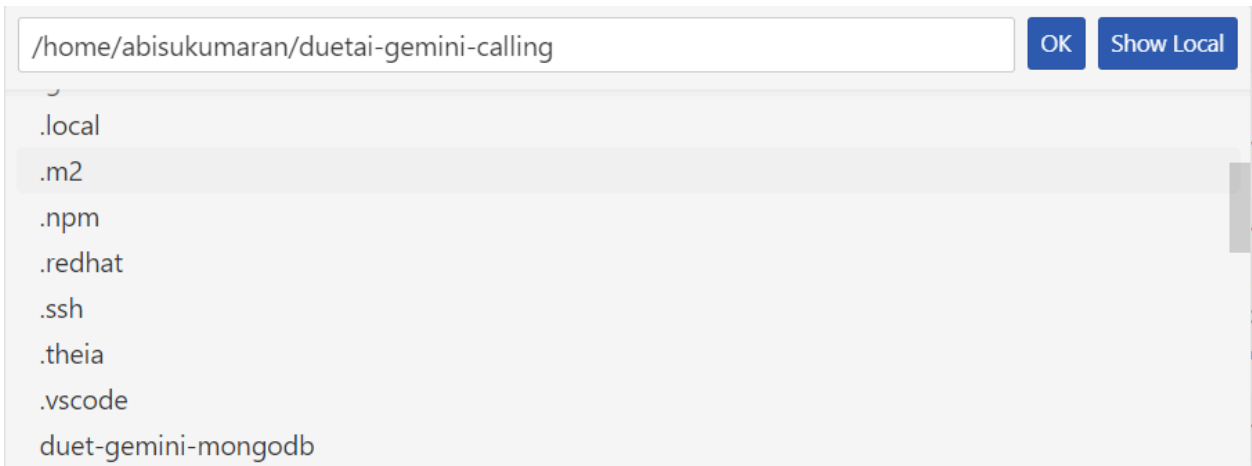
兩者都啟用後，按一下左下角的專案名稱，然後在開啟的彈出式清單中，向下捲動至「New Application」(新應用程式)。



在該清單中，選取「Cloud Functions application」(Cloud Functions 應用程式)。在彈出式清單中選取 Java：

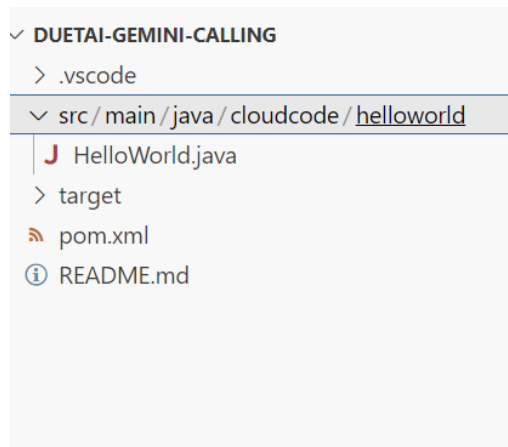


在產生的清單中，輸入專案名稱「duetai-gemini-calling」而非 helloworld，然後按一下「確定」。



太棒了！您已使用 Gemini 啟動簡單的 Java Cloud Functions 應用程式，除了啟用和啟動設定外，您幾乎沒有做任何事，對吧？

您應該會看到以下專案結構：



現在可以部署函式了。但這並非我們啟動這項計畫的原因。接下來，我們將使用 Java SDK，在這個 Cloud Function 中建構 Gemini Pro API 實作項目。

現在，讓我們為這個用途建構功能，也就是在這個 Cloud Function 中叫用 Gemini Pro 模型。為此，您可以自由新增更多提示，讓 Gemini 逐步開發程式碼，或自行編寫邏輯。我會混合使用這兩種方法。

步驟 6 · 約 0 分鐘

6. 新增依附元件

在 Gemini 對話控制台 (位於左窗格的 Cloud Code 編輯器內) 中，輸入下列提示：

```
what is the maven dependency for com.google.cloud.vertexai library
```

我要求提供 com.google.cloud.vertexai 套件的具體範例，是因為我在實作 Gemini 呼叫程式碼時，就是使用這個套件。

我得到以下結果：

```
what is the maven dependency for com.google.cloud.vertexai library
```

The maven dependency for com.google.cloud.vertexai library is:

```
<dependency>
  <groupId>com.google.cloud</groupId>
  <artifactId>google-cloud-vertexai</artifactId>
  <version>2.12.0</version>
</dependency>
```

Generated code may be subject to license restrictions not shown here. Use code with care.

[Learn more](#)

This dependency will provide you with the necessary APIs to interact with the Vertex AI API.

Enter a prompt here

```
<dependency>
  <groupId>com.google.cloud</groupId>
  <artifactId>google-cloud-vertexai</artifactId>
  <version>0.1.0</version>
</dependency>
```

複製這段程式碼，然後貼到 pom.xml 檔案中，緊接在 **</dependencies>** 標記之前。將版本替換為 0.1.0 (如果您使用 Spring Cloud GCP BOM 管理 spring-cloud-gcp 版本號碼，可以移除 **<version>** 標記)。

依附元件部分應如下所示：

 pom.xml

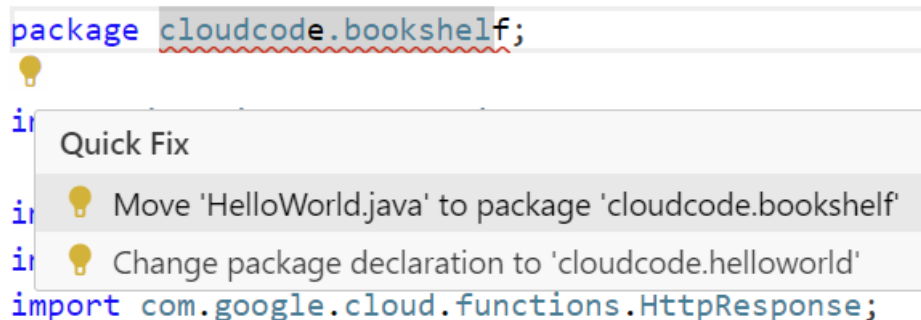
```
20     <version>1.1.0</version>
21   </dependency>
22
23   <dependency>
24     <groupId>com.google.cloud</groupId>
25     <artifactId>google-cloud-vertexai</artifactId>
26     <version>0.1.0</version>
27   </dependency>
28
29   <dependency>
30     <groupId>com.google.code.gson</groupId>
31     <artifactId>gson</artifactId>
32     <version>2.10</version>
33   </dependency>
34
35 </dependencies>
```

請務必視需要更新版本號碼，使其與上述內容相符。如您所見，我還一併加入了另一個依附元件：

```
<dependency>
  <groupId>com.google.code.gson</groupId>
  <artifactId>gson</artifactId>
  <version>2.10</version>
</dependency>
```

7. 修改函式進入點和類別名稱

1. 前往「.vscode」資料夾下的「launch.json」檔案。將函式名稱從「function-hello-world」編輯為「function-gemini-calling」。
2. 將 entryPoint 值從「cloudcode.helloworld.HelloWorld」更新為「cloudcode.bookshelf.Bookshelf」。
3. 現在請前往「HelloWorld.java」Java 類別檔案。將套件名稱變更為套件 cloudcode.bookshelf；在彈出的錯誤中，按一下黃色燈泡，然後按一下「Move HelloWorld.java」選項，將套件移至 cloudcode.bookshelf。



4. 將類別名稱更新為 Bookshelf，然後在彈出的錯誤訊息中，按一下黃色小燈泡，並選取「Rename file to Bookshelf.java」（將檔案重新命名為 Bookshelf.java）。選取該選項。
-

步驟 8 · 約 0 分鐘

8. 建立呼叫 Gemini Pro 的方法

我們在 Bookshelf.java 類別中實作這項功能。將 Bookshelf.java 替換為下列程式碼：

```

package cloudcode.bookshelf;
import java.io.BufferedWriter;
import com.google.cloud.functions.HttpFunction;
import com.google.cloud.functions.HttpRequest;
import com.google.cloud.functions.HttpResponse;
import com.google.cloud.vertexai.VertexAI;
import com.google.cloud.vertexai.api.GenerateContentResponse;
import com.google.cloud.vertexai.api.GenerationConfig;
import com.google.cloud.vertexai.generativeai.preview.GenerativeModel;
import com.google.cloud.vertexai.generativeai.preview.ResponseHandler;
import java.io.IOException;
import java.util.List;
import java.util.Arrays;
import java.util.Map;
import java.util.LinkedHashMap;
import com.google.gson.Gson;
import com.google.gson.JsonObject;
import com.google.gson.JsonArray;

public class Bookshelf implements HttpFunction {
    private static final Gson gson = new Gson();

    @Override
    public void service(HttpRequest request, HttpResponse response) throws Exception {
        BufferedWriter writer = response.getWriter();

        // Get the request body as a JSON object.
        JsonObject requestJson = new Gson().fromJson(request.getReader(), JsonObject.class);
        JsonArray calls_array = requestJson.getAsJsonArray("calls");
        JsonArray calls = (JsonArray) calls_array.get(0);
        String context = calls.get(0).toString().replace("\\\"", "");

        //Invoke Gemini model
        String raw_result = callGemini(context);
        raw_result = raw_result.replace("\n","");
        String trimmed = raw_result.trim();
        List<String> result_list = Arrays.asList(trimmed);
        Map<String, List<String>> stringMap = new LinkedHashMap<>();
        stringMap.put("replies", result_list);

        // Serialization
        String return_value = gson.toJson(stringMap);
        writer.write(return_value);
    }

    public String callGemini(String context) throws IOException{
        String res = "";
        try (VertexAI vertexAi = new VertexAI("REPLACE_WITH_YOUR_PROJECT_ID", "us-central1");
        ) {
            GenerationConfig generationConfig =
                GenerationConfig.newBuilder()
                    .setMaxOutputTokens(2048)
                    .setTemperature(0.4F)

```

```

        .setTopK(32)
        .setTopP(1)
        .build();
    GenerativeModel model = new GenerativeModel("gemini-pro", generationConfig,
vertexAi);
    GenerateContentResponse response = model.generateContent(context);
    res = ResponseHandler.getText(response);
} catch (Exception e) {
    System.out.println(e);
}
    return res;
}
}

```

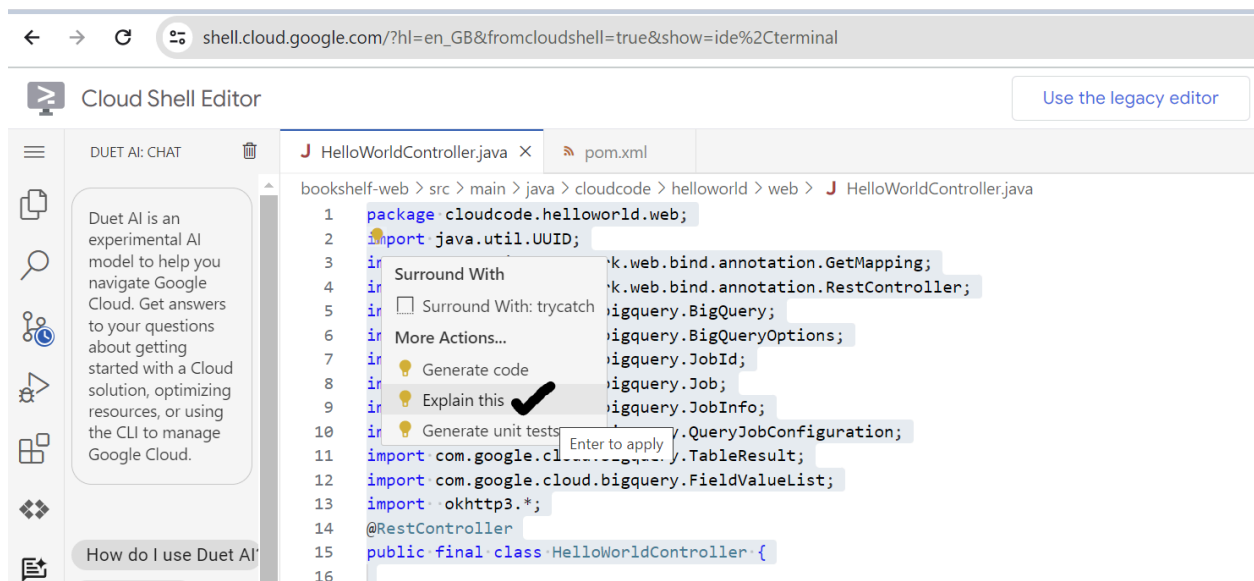
這個類別預期輸入的 JSON 結構如下：

```
{ "calls": [ ["YOUR_PROMPT_HERE"] ] }
```

系統會傳回如下所示的回應：

```
(Json) Map<String, List<String>> {"replies": ["response"]}
```

在左側窗格的 Cloud Shell 編輯器中，試用 Gemini 對話選項來解說程式碼。或者，您也可以選取所有程式碼，然後按一下選取範圍左上角的黃色燈泡，並選擇「說明這項內容」選項。



9. 部署 Cloud 函式

Cloud 函式已準備就緒，現在來問問 Gemini 如何部署。前往 Cloud Code 編輯器中的 Gemini 對話，然後輸入下列內容：

How to deploy this Cloud Function with a gcloud command?

我收到以下回覆：

gcloud functions deploy YOUR_FUNCTION_NAME \
--runtime java17 \
--trigger-http \
--allow-unauthenticated

Where YOUR_FUNCTION_NAME is the name of your Cloud Function.

For more information on deploying Cloud Functions, please see the [Cloud Functions documentation](#).

Rate this answer:

我現在想進一步探究。因此我請 Gemini 提供完整的 gcloud functions deploy 指令。回應如下所示：

CLOUD SHELL
Editor

Open Terminal | Use the legacy editor

DUET AI: CHAT

Give me the full gcloud functions deploy command

result_list

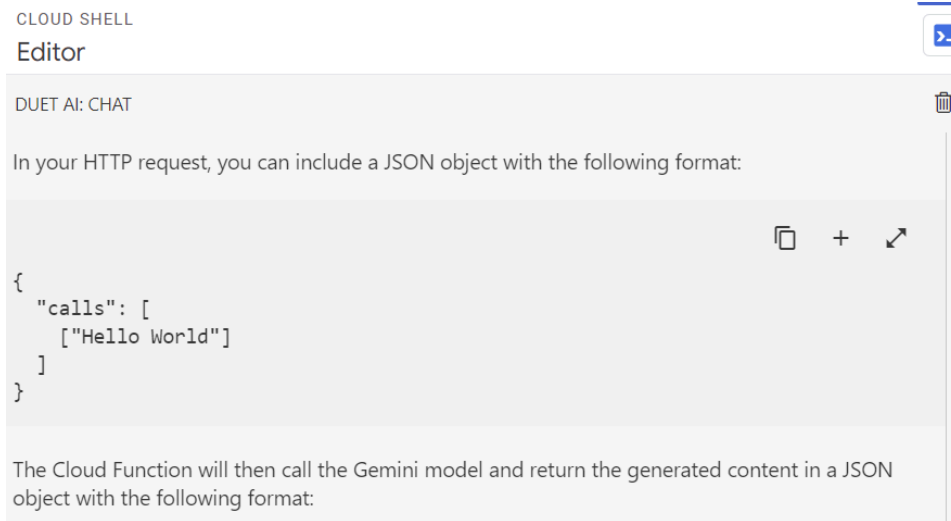
The full gcloud functions deploy command is:

gcloud functions deploy bookshelf --runtime java17 --trigger-http --entry-point cloudcode.bookshelf.Bookshelf

Explain this

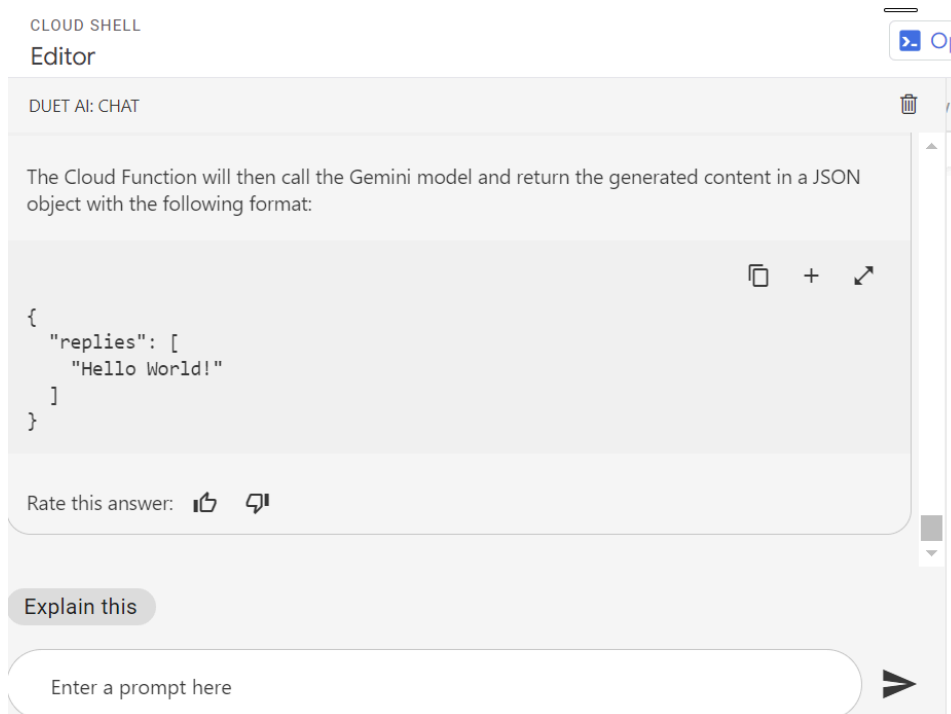
我無法保證你會收到相同回覆，但令我驚訝的是，Gemini 竟然還補充了更多細節，如下圖所示：

要求主體格式：



和

回覆格式：



現在請執行 Gemini 提供的 gcloud 指令，部署函式。為此，我們需要開啟 Cloud Shell 終端機。您可以在新分頁中開啟 <https://console.cloud.google.com>，並確認已選取正確的專案。按一下控制台右上角的「啟用 Cloud Shell」圖示，開啟 Cloud Shell 終端機，然後使用下列指令，確認您位於正確的專案資料夾：

```
cd duetai-gemini-calling
```

然後執行下列指令：

```
gcloud functions deploy bookshelf --runtime java17 --trigger-http --entry-point  
cloudcode.bookshelf.Bookshelf --allow-unauthenticated
```

系統會詢問「允許未經驗證就调用新函式 [bookshelf] 嗎？」說出「y」並按下 Enter 鍵。之後系統會詢問幾個問題 (如有)，並部署無伺服器 Cloud Function，以及部署的網址：https://us-central1-*****.cloudfunctions.net/bookshelf。

現在，讓我們叫用已部署的 Cloud Functions 並進行測試！

注意：如果您不小心略過「允許未經驗證的叫用」問題或選取「N」，則未授予額外 IAM 設定時，您將無法存取 Cloud Functions 的結果，並會看到「權限錯誤」。請注意這一點。

步驟 10 · 約 0 分鐘

10. 呼叫已部署的 Cloud 函式

我們來問問 Gemini 吧。我輸入了提示

How to call the deployed cloud function?

我得到以下結果：(您可能會看到完全相同的結果，也可能不會。歡迎隨意調整提示，看看回覆內容有何不同)。

DUET AI: CHAT

To call the deployed Cloud Function, you can use the following steps:

1. Open the Cloud Functions page in the Google Cloud console.
2. Click on the name of the Cloud Function that you want to call.
3. Click on the **Trigger** tab.
4. Copy the URL that is displayed in the **URL** field.
5. Paste the URL into a web browser and press Enter.

This will send an HTTP request to the Cloud Function and return the response.

In the case of the Cloud Function that you deployed, the response will be a JSON object with the following format:

```
{
  "replies": [
    "Hello World!"
  ]
}
```

Explain this

針對部署的函式，詢問其他叫用方式、使用 gcloud 指令呼叫等問題。我提交了以下提示：

how to call the deployed cloud function using gcloud

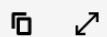
我收到以下回覆：

how to call the deployed cloud function using gcloud

To call a deployed Cloud Function using gcloud, you can use the following steps:

1. In the Google Cloud console, activate Cloud Shell.
2. Run the following command:

```
gcloud functions call <function-name> --data '{"message": "Hello world!"}'
```



您可以從終端機使用這項回應 (「gcloud functions call」指令)，並進行調整，使其適用於我們的情境 (或者，嘗試在提示中傳遞參數，看看是否能在回應中取得詳細的 gcloud functions call)：

```
gcloud functions call bookshelf --region=us-central1 --gen2 --data '{"calls":[{"Hello! This is my test prompt."}]}'
```

我的結果如下：

```
abisukumaran@cloudshell:~/duet-ai-gemini-calling (abis-345004)$ gcloud functions call bookshelf --region=us-central1 --gen2 --data '{"ca
test prompt."}]}'

{"replies":["Sure, here is your test prompt:You are a marketing consultant for
a small business that sells handmade jewelry. The business is struggling to increase
sales and reach new customers. Your task is to develop a comprehensive marketing
plan that will help the business achieve its goals.Your marketing plan should include
the following elements:* **Target audience:** Who is the ideal customer for this
business?* **Marketing objectives:** What does the business hope to achieve with
its marketing efforts?* **Marketing strategies:** What specific tactics will be
used to reach the target audience and achieve the marketing objectives?* **Marketing
budget:** How much money is available to spend on marketing?* **Marketing measurement:**
How will the success of the marketing plan be measured?In addition to these elements,
your marketing plan should also be creative and innovative. The handmade jewelry
business is a unique and special one, and your marketing plan should reflect that.Here
are some additional tips for developing your marketing plan:* **Do your research.**
Before you can develop a marketing plan, you need to understand the business and
its target audience. This means conducting market research and gathering data on
your customers.* **Set realistic goals.** Don't try to do too much with your
marketing plan. Set realistic goals that you can achieve with the resources you
have available.* **Be creative.** The handmade jewelry business is a creative one,
so your marketing plan should be too. Think outside the box and come up with unique
and innovative ways to reach your target audience.* **Measure your results.** It's
important to track the results of your marketing efforts so you can see what's
working and what's not. This will help you improve your plan over time.I hope
this helps! Please let me know if you have any other questions."}]}'
abisukumaran@cloudshell:~/duet-ai-gemini-calling (abis-345004)$
```

步驟 11 · 約 0 分鐘

11. 清除所用資源

如要刪除先前建立的 Cloud Functions，請按一下 Cloud Functions 詳細資料頁面中的「DELETE」按鈕。

12. 恭喜

您已成功建構、部署及測試 Java Cloud Functions，並使用 Gemini 呼叫 Gemini 1.0 Pro！這項應用程式會根據書籍摘要和主題，取得與書籍推薦相關的輸入提示。
